

Iterables, Iterators, Generator expressions, Generators (and list comprehensions)

1. Iterables and Iterators

Iterables

```
1 from collections import Counter
2 from collections.abc import Iterable
3
4 def frequent_letters(sentences: Iterable[str]) -> list[tuple]:
5     c = Counter()
6     for sentence in sentences:
7         c.update(sentence)
8     return c.most_common(5)
9
10 print(frequent_letters(["hello there world!", "hello!"]))
11 print(frequent_letters(("hello there world!", "hello!")))
12 print(frequent_letters({"hello there world!", "hello!"}))
13 print(frequent_letters("hello"))
```

```
[('l', 5), ('e', 4), ('h', 3), ('o', 3), (' ', 2)]
[('l', 5), ('e', 4), ('h', 3), ('o', 3), (' ', 2)]
[('l', 5), ('e', 4), ('h', 3), ('o', 3), (' ', 2)]
[('l', 2), ('h', 1), ('e', 1), ('o', 1)]
```

Iterables

```
1 from collections import Counter
2 from collections.abc import Iterable
3
4 def frequent_letters(sentences: Iterable[str]) -> list[tuple]:
5     c = Counter()
6     for sentence in sentences:
7         c.update(sentence)
8     return c.most_common(5)
9
10 print(frequent_letters(2026))
```

Traceback (most recent call last):

File "/Users/mask3308/undervisning/tillämpad programmering

print(frequent_letters(2026))

~~~~~

File "/Users/mask3308/undervisning/tillämpad programmering

for sentence in sentences:

~~~~~

TypeError: 'int' object is not iterable

Iterable and Iterator is not the same

Every Iterator is also an Iterable,

But there are a lot of Iterables that are not Iterators (e.g. Lists are not Iterators).

Here, we take a list and make it into an Iterator. Then, the function `next()` can be used to get the next element.

```
1 my_list = ["Hello there!", "What's up!", "Goodbye"]
2 my_iterator = iter(my_list)
3
4 print(next(my_iterator))
5 print(next(my_iterator))
6 print(next(my_iterator))
```

Hello there!

What's up!

Goodbye

Iterable and Iterator is not the same

If it is not an Iterator, we can't run next:

```
1 my_list = ["Hello there!", "What's up!", "Goodbye"]
2 print(next(my_list))
```

Traceback (most recent call last):

```
File "/Users/mask3308/undervisning/tillämpad programmering
  print(next(my_list))
    ~~~~~
```

TypeError: 'list' object is not an iterator

An iterator can only be used once

A normal list (i.e., **not** an Iterator).

What will be printed here?

```
1 my_list = ["Hello there!", "What's up!", "Goodbye, then"]
2 for sentence in my_list:
3     print(sentence)
4 print("---")
5 for sentence in my_list:
6     print(sentence)
```

An iterator can only be used once

A normal list (i.e., **not** an Iterator).

What will be printed here?

```
1 my_list = ["Hello there!", "What's up!", "Goodbye, then"]
2 for sentence in my_list:
3     print(sentence)
4 print("---")
5 for sentence in my_list:
6     print(sentence)
```

Hello there!

What's up!

Goodbye, then

Hello there!

What's up!

Goodbye, then

An iterator can only be used once

An Iterator:

What will be printed here?

```
1 my_list = ["Hello there!", "What's up!", "Goodbye, then"]
2 my_iterator = iter(my_list)
3
4 for sentence in my_iterator:
5     print(sentence)
6 print("---")
7 for sentence in my_iterator:
8     print(sentence)
```

An iterator can only be used once

An Iterator:

What will be printed here?

```
1 my_list = ["Hello there!", "What's up!", "Goodbye, then"]
2 my_iterator = iter(my_list)
3
4 for sentence in my_iterator:
5     print(sentence)
6 print("---")
7 for sentence in my_iterator:
8     print(sentence)
```

Hello there!

What's up!

Goodbye, then

An iterator can only be used once

```
1 my_list = [1, 2, 3]
2 my_iterator = iter(my_list)
3 print(my_iterator)
4 print(sum(my_iterator))
```

```
<list_iterator object at 0x1018d9480>
```

```
6
```

```
0
```

An iterator can only be used once

`next()` can be given a value to return, when the iterator has ended.
(Here `None`)

```
1 my_list = [1, 2, 3]
2 my_iterator = iter(my_list)
3
4 next_value = next(my_iterator, None)
5 while next_value:
6     print(next_value)
7     next_value = next(my_iterator, None)
```

```
1
2
3
```

2. List comprehensions

List comprehensions

A list comprehension that does nothing:

```
1 greeting = ["Hello", "there", "world", "!"]  
2 print([el for el in greeting])
```

```
['Hello', 'there', 'world', '!']
```

List comprehensions

A list comprehension that counts length of each word:

```
1 greeting = ["Hello", "there", "world", "!"]  
2 print([len(el) for el in greeting])
```

[5, 5, 5, 1]

List comprehensions

We can also sum the lengths:

```
1 greeting = ["Hello", "there", "world", "!"]
2 lengths = [len(el) for el in greeting]
3 print(lengths)
4 print("Sum:", sum(lengths))
```

[5, 5, 5, 1]

Sum: 16

3. Generator expressions

Problem

We create an extra list of integers, that is not really needed. We only need the final sum of the lengths.

```
1 greeting = ["Hello", "there", "world", "!"]
2 lengths = [len(el) for el in greeting]
3 print(lengths)
4 print("Sum:", sum(lengths))
```

[5, 5, 5, 1]

Sum: 16

It doesn't matter for a small list.

But what if the list would be really long? Then it would take up a lot of memory.

Generator expressions

```
1 greeting = ["Hello", "there", "world", "!"]
2 lengths = [len(el) for el in greeting]
3 print(lengths)
4 print("Sum:", sum(lengths))
```

```
[5, 5, 5, 1]
```

```
Sum: 16
```

Instead of a list comprehension, we can use a generator expression:

```
1 greeting = ["Hello", "there", "world", "!"]
2 lengths = (len(el) for el in greeting)
3 print(lengths)
4 print("Sum:", sum(lengths))
```

Then, no list with numbers is produced, but a generator:

```
<generator object <genexpr> at 0x10375d560>
```

```
Sum: 16
```

next

A generator only returns one element at a time, and in a **lazy** way. That is, only when asked for it.

By calling `next()`, we can use the generative functionality more explicitly:

```
1 greeting = ["Hello", "there", "world", "!"]
2 lengths = (len(el) for el in greeting)
3 next_value = next(lengths, None)
4 while next_value:
5     print(next_value)
6     next_value = next(lengths, None)
```

5
5
5
1

A generator is a kind of iterator

A generator is a kind of iterator. Remember: Iterators can only be used once.

```
1 my_list = [1, 2, 3]
2 my_iterator = iter(my_list)
3 print(my_iterator)
4 print(sum(my_iterator))
```

```
<list_iterator object at 0x1018d9480>
```

```
6
```

```
0
```

A generator can only be used once

A generator is a type of Iterator. So a generator **can only be used once**:

```
1 greeting = ["Hello", "there", "world", "!"]
2 lengths = (len(el) for el in greeting)
3 print("Sum:", sum(lengths))
4 print("Sum:", sum(lengths))
```

Sum: 16

Sum: 0

A generator can only be used once

If using `next()`, we get an exception, when we reach the end of the iterator.

```
1 greeting = ["Hello", "there", "world", "!"]
2 lengths = (len(el) for el in greeting)
3 print("Sum:", sum(lengths))
4 print(next(lengths))
```

Sum: 16

Traceback (most recent call last):

File "<python-input-19>", line 1, in <module>

print(next(lengths))

~~~~~

StopIteration

# A generator can only be used once

If we want to be able to use our generator several times, then we can turn it into a list.

(But then, we will not achieve the intended memory saving, so we could just have made a list comprehension to start with.)

---

```
1 greeting = ["Hello", "there", "world", "!"]
2 lengths = (len(el) for el in greeting)
3 lengths = list(lengths)
4 print(lengths)
5 print("Sum:", sum(lengths))
6 print("Sum:", sum(lengths))
```

---

[5, 5, 5, 1]

Sum: 16

Sum: 16



# Writing our own generator functions

What if we want to write our own generator function?

Start writing a function that does the same as the list comprehension:

---

```
1 def get_lengths_for_words(words):
2     lengths = []
3     for el in words:
4         lengths.append(len(el))
5     return lengths
6
7 greeting = ["Hello", "there", "world", "!"]
8 lengths = get_lengths_for_words(greeting)
9 print(lengths)
10 print("Sum: ", sum(lengths))
```

---

[5, 5, 5, 1]

Sum: 16

# Writing our own generator functions

## Old version:

---

```
1 def get_lengths_for_words(words):
2     lengths = []
3     for el in words:
4         lengths.append(len(el))
5     return lengths
```

---

## Generator version:

---

```
1 def get_lengths_space_saving(words):
2     for el in words:
3         yield len(el)
4
5 greeting = ["Hello", "there", "world", "!"]
6 lengths = get_lengths_space_saving(greeting)
7 print(lengths)
8 print("Sum: ", sum(lengths))
```

---

```
<generator object get_lengths_space_saving at 0x101ae6670>
Sum: 16
```

# Writing our own generator functions

The values from the generator are retrieved in a **lazy** fashion, i.e., not until they are actually needed. And our generator functions automatically saves the state in the loop for us.

---

```
1 def get_lengths_space_saving(words):
2     for el in words:
3         yield len(el)
4
5 greeting = ["Hello", "there", "world", "!"]
6 lengths = get_lengths_space_saving(greeting)
7 next_value = next(lengths, None)
8 while next_value:
9     print(next_value)
10    next_value = next(lengths, None)
```

---

5  
5  
5  
1

# Infinite generators

---

```
1 def infinite_counting():
2     n = 0
3     while True:
4         yield n
5         n += 1
6
7 counting = infinite_counting()
8 while True:
9     print(next(counting))
```

---

0

1

2

3

...

# Stop generating content from infinite generators

Even if the generator is infinite (or very long), you can stop generating content from it earlier

---

```
1 def infinite_counting():
2     n = 0
3     while True:
4         yield n
5         n += 1
6
7 counting = infinite_counting()
8 count_result = next(counting)
9 while count_result < 3:
10     print(count_result)
11     count_result = next(counting)
```

---

0

1

2